Table des matières

Exercices corrigés 12 — Rebâtir le pipeline de bout en bout 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 12 — Rebâtir le pipeline de bout en bout

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur la plateforme que vous venez d'effacer et de refaire, et sur les chiffres que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 12 et du Labo 12, avec quelques retours aux cours précédents.

Les chiffres à retrouver. Les questions pratiques supposent le labo fini : l'entrepôt d'avant dans `plateforme/avant/`, l'entrepôt refait par le DAG, le chemin chaud rejoué de zéro. Les requêtes DuckDB se tapent dans la ligne de commande DuckDB, lancée depuis `Projet_Boreal_intermediaire`, avec les deux entrepôts ouverts en lecture seule quand la question compare : `ATTACH 'plateforme/avant/boreal.duckdb' AS avant (READ_ONLY); ATTACH 'plateforme/entrepot/boreal.duckdb' AS refait (READ_ONLY);`. Les requêtes de la base de séries temporelles se tapent dans `psql`, depuis Ubuntu, dans `plateforme`, après `SET TIME ZONE 'America/Toronto';`.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Fini. Nommez les trois promesses d'une plateforme finie, et l'épreuve du capstone qui met chacune à l'essai.
2. De zéro. Dites ce que la reconstruction du capstone efface, ce qu'elle garde, et pourquoi une entreprise qui ne garde pas ses livraisons ne peut pas faire la même épreuve.

© 2026 iAlexMG. Tous droits réservés.

3. Trois familles. Rangez dans sa famille, versionné par vous, reçu d'un autre ou dérivé, chacun de ces objets : `contrats/argent_usine.yaml`, `sources/usine/historien/`, `plateforme/lac/bronze/`, `plateforme/docker-compose.yml`, `grafana/usine.json`, `pipeline.controles`, `01_de/.env`.
4. Ce qui ne revient pas. Donnez trois choses qu'une reconstruction ne peut pas refaire, et ce qui les garde quand on en a besoin.
5. Sans horaire. Dites ce que font `schedule=None` et `max_active_runs=1` dans le DAG de reconstruction, et pourquoi ils conviennent à ce DAG et pas au quotidien.
6. Flèches et pool. Les tâches `erp` et `meteo` n'ont aucune flèche, et pourtant elles ne tournent jamais en même temps que `bronze`. Expliquez les deux faits.
7. Le fichier attaché deux fois. Dites pourquoi `argent_usine.sql`, lancé tel quel par le pipeline, échoue sur `Unique file handle conflict`, et pourquoi l'étape adapte le script plutôt que d'en garder une seconde copie.
8. Le pilote dans l'image. Pourquoi le pilote ODBC s'installe-t-il dans le `Dockerfile.airflow` plutôt qu'au démarrage du conteneur, et pourquoi en `root` ?
9. Deux adresses pour une base. Sous Windows, le pipeline joint la base de l'ERP par `localhost:5433` ; dans le conteneur d'Airflow, par `erp:5432`. Expliquez les deux, et dites par quel mécanisme du Cours 05 le même code prend la bonne.
10. Le fuseau de la session. Expliquez pourquoi le contrat de l'argent, jugé sur avril 2026 dans le conteneur, compte 29 jours manquants et non 30.
11. Un chiffre du milieu. Pourquoi la correction de la fenêtre et la règle `heures_du_jour_livrees` font-elles passer le contrat de l'argent en 1.1.0, et non en 2.0.0 ?
12. Trois preuves. Pour chacune des trois preuves de la reconstruction, dites ce qu'elle compare, ce qu'elle voit et ce qu'elle ne voit pas.
13. Mettre de côté. Pourquoi le labo déplace-t-il l'entrepôt dans `plateforme/avant/` plutôt que de le supprimer ? Que resterait-il comme preuve sans ce geste ?
1. La même minute. Pourquoi l'alerte de vibration revient-elle le 2025-10-22 à 8 h 15, un autre jour, sur une plateforme en partie refaite ? Quelle colonne change, et pourquoi ?
2. Qui fait quoi dans Grafana. Dites ce que Data Engineering livre avec le tableau de bord, ce qui appartient à Business Intelligence, et ce que fait le réglage `allowUiUpdates: false`.

Questions pratiques

1. [labo] Le coût de la traversée. Écrivez la requête qui donne, dans l'entrepôt d'avant et dans le refait, la durée de chaque passage de l'étape `argent`. Dites d'où vient l'écart.
2. [labo] L'histoire que le journal garde. Écrivez la requête qui donne, dans chacun des deux journaux des chargements, les fichiers chargés plus d'une fois, avec le nombre de lignes de chaque version.

© 2026 iAlexMG. Tous droits réservés.

3. [labo] La fraîcheur de l'argent, mois par mois. Rejouez le contrat de l'argent mois par mois, d'octobre 2025 à avril 2026, avec le rejoueur du Cours 11. Dites quels mois rompent quelle règle.
4. [labo] Les jours qui n'ont pas 24 heures. Écrivez la requête qui donne les jours de Boréal de mars 2026 dont `argent.usine_horaire` n'a pas 24 heures, puis la même en jours UTC. Expliquez la différence.
5. [labo] Le journal sans son état. Comparez les deux journaux des chargements par `EXCEPT ALL`, sans la ligne `ETAT`, dans les deux sens. Nommez les lignes qui sortent.
6. [labo] Le volume de points, vu du chemin chaud. Dans `psql`, comptez les points de `UTL.AIR.PRESSION` par jour, du 13 au 24 octobre 2025. Reliez ce que vous voyez au Cours 11.
7. [labo] Où passe le temps d'Airflow. À partir de `pipeline.executions` de l'entrepôt refait, donnez la somme des durées des sept étapes et le temps écoulé entre le début de la première et la fin de la dernière. Comparez à la durée de l'exécution que donne Airflow, et expliquez.
8. [labo] Le bulletin a-t-il été suivi ? Comptez, dans `raw.garanties_visites`, les visites des motoneiges de la plage `MN26001238` à `MN26001466`, par type de visite, avant et après le 2026-02-16.
9. [labo] La fenêtre de la météo. La tâche `meteo` de la reconstruction a tiré 92 jours de l'API d'ECCC, en deux pages. Retrouvez d'où viennent ces trois nombres, et le fichier qu'elle a écrit.
10. [labo] Un panneau de plus. Écrivez la requête d'un panneau de Grafana qui compte, par jour de Boréal, les points de pression reçus dans `temps_reel.points`. Vérifiez-la dans `psql` sur les 18 à 22 octobre 2025, puis comparez avec `time_bucket('1 day', instant)` sans fuseau.

Diagnostics

1. Pressé, un collègue tape `rmdir /s /q plateforme\entrepot` à l'étape 4, au lieu de déplacer le dossier. La reconstruction passe au vert. Que peut-il encore prouver, et que ne peut-il plus ?
2. Une développeuse trouve plus simple de mettre `argent` dans « tout » : troisième champ à `True` dans `ETAPES`. Qu'est-ce qui change pour le kit de départ d'Analytics Engineering, et pour le sens de « tout » ?
3. Pour accélérer la reconstruction, un ingénieur retire `pool=ENTREPOT` des tâches `erp` et `meteo`, qui ne dépendent de rien. Qu'arrive-t-il au prochain déclenchement ?
4. Plutôt que de retoucher le contrat, une collègue ajoute `TZ: America/Toronto` à l'environnement des conteneurs d'Airflow. Le contrat rend alors les bons verdicts dans le conteneur. Pourquoi la correction du labo reste-t-elle la bonne ?
5. Pour gagner du temps, `comparer_reconstruction.py` ne calcule plus que `avant EXCEPT ALL refait`. Donnez un défaut de reconstruction que le script ne verrait plus.

© 2026 iAlexMG. Tous droits réservés.

6. Un analyste améliore le tableau de bord dans l'interface de Grafana, puis l'enregistre sous un autre nom pour contourner le refus. Quels sont les problèmes ?

7. À l'étape 7, un lecteur oublie le `TRUNCATE` et lance directement les trois programmes et le rejeu. Que disent les bilans, et l'alerte se lève-t-elle ?

8. Un collègue veut retrouver l'alerte dans Grafana, sans le programme du Cours 09 : la moyenne de `UTL.CMP2.VIBRATION` par `time_bucket('15 minutes', instant)`, au-dessus de 4,5. Il la voit franchie dès la fenêtre de 7 h 45 le 22 octobre, et en conclut que l'alerte du Cours 09 retarde d'un quart d'heure. Qu'est-ce qui cloche ?

Questions à développer

A. « Une plateforme de données se sauvegarde ; la reconstruire de zéro est un exercice d'école. » Discutez, avec ce que le capstone a trouvé en reconstruisant, et ce qu'une sauvegarde garde qu'une reconstruction ne garde pas.

B. Le chemin froid se reconstruit, le chemin chaud se rejoue. Expliquez la différence, ce que la rétention de Kafka change à la seconde, et ce qu'il faudrait garder pour rejouer l'usine d'il y a deux ans.

C. L'incident de l'automne 2025 se lit de bout en bout dans ce que Data Engineering livre, et pourtant aucune requête du labo ne joint deux tables. Dites ce que Data Engineering doit à l'analyse d'un incident, où il s'arrête, et pourquoi.

Mises en situation

1. Une laiterie fictive de Saint-Hyacinthe fait tourner sa plateforme de données depuis trois ans sur un serveur que son unique ingénieur de données a monté seul. Il part dans deux mois. La directrice vous demande de « s'assurer que ça continue de marcher ». Proposez un plan, avec ce qu'il faut écrire, ce qu'il faut effacer, dans quel ordre, et ce qui prouvera que c'est fait.

2. Une minière fictive de Val-d'Or fait tourner Airflow sur un serveur Linux réglé en UTC. Ses contrats de données ont été écrits et essayés par des analystes sur leurs portables, à l'heure de l'Est. Depuis la mise en service, le contrat du tonnage quotidien rompt une nuit sur trois, jamais sur les portables. Menez le diagnostic et proposez la correction.

3. Une coopérative de pêche fictive de Gaspé a un tableau de bord de débarquements, monté à la souris dans l'interface de Grafana par une analyste. La plateforme doit changer de serveur le mois prochain. L'analyste propose d'exporter le tableau de bord la veille et de le réimporter. Donnez votre avis et votre proposition.

4. (avancé) Un distributeur fictif de Joliette a refait son entrepôt de zéro et comparé les lignes dans les deux sens. Le rapport donne 3 écarts : 2 dans le journal des chargements, dans le sens « avant moins refait », et 1 dans la table des factures, dans le sens « refait moins avant ». Classez chaque écart, dites ce que vous vérifiez pour chacun, et ce que vous répondez à qui demande si la reconstruction est prouvée.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire le corrigé. Chaque réponse finit par le renvoi à la section de la théorie, ou à l'étape du labo, qui porte la matière. Les chiffres viennent du Labo 12 mené jusqu'au bout sur la génération jusqu'à 2026-03 : un autre ordre de gestes change des durées et des comptes de lignes d'exécution, jamais les données.

Questions courtes

1. Fini. Elle se refait : le dépôt, les sources et la plateforme suffisent ; l'épreuve est d'effacer le lac et l'entrepôt puis de tout relancer par un seul DAG. Elle se prouve : ce qui revient égale ce qui était là ; l'épreuve, ce sont les comptes, les lignes dans les deux sens et les contrats. Elle se passe : quelqu'un d'autre la reprend sans vous ; l'épreuve, c'est le `README.md` du dépôt, `LIVRAISON.md`, les contrats et l'étiquette `v1.0`. *Théorie : `Theorie_12_DE.md`, section 1.*

2. De zéro. Elle efface le lac et l'entrepôt, tout ce que Data Engineering produit. Elle garde les sources, le dépôt et la plateforme avec ses services. Les sources jouent le rôle des livraisons gardées telles qu'elles sont arrivées : sans elles, rien ne se recharge, et il ne reste qu'à restaurer une sauvegarde en espérant qu'elle est bonne. *Théorie : `Theorie_12_DE.md`, section 1.*

3. Trois familles. Versionnés par vous : `contrats/argent_usine.yaml`, `grafana/usine.json`. Reçus d'un autre : `sources/usine/historien/`, livré par l'historien de l'usine ; `plateforme/docker-compose.yml`, tenu par l'équipe de la plateforme ; `01_de/.env`, un secret qu'on remplit à la main depuis son modèle et qui ne voyage jamais. Dérivés : `plateforme/lac/bronze/` et `pipeline.controles`, qui se refont à tout moment. *Théorie : `Theorie_12_DE.md`, section 3.*

4. Ce qui ne revient pas. Les versions remplacées d'un fichier livré de nouveau sous le même nom : l'export de la GMAO à 779 bons et le catalogue à 33 documents, que le journal d'avant raconte ; les exécutions et les verdicts d'hier ; l'heure de réception des messages du chemin chaud. Ce qui les garde, c'est une sauvegarde de l'entrepôt et des journaux, pas le dépôt. *Théorie : `Theorie_12_DE.md`, section 3.*

5. Sans horaire. `schedule=None` : aucune exécution planifiée, le DAG ne part que si on le déclenche. `max_active_runs=1` : une seule exécution à la fois. Une reconstruction se lance quand on veut prouver que le dépôt suffit, et deux en même temps n'auraient aucun sens. Le quotidien, lui, a une tranche de temps à traiter chaque nuit : il lui faut un horaire et un intervalle de données (Cours 06). *Théorie : `Theorie_12_DE.md`, section 4.*

6. Flèches et pool. Aucune flèche, parce que ni l'ERP ni la météo ne nourrissent la `RAW` ou l'argent : les flèches disent les dépendances des données, et il n'y en a pas. Jamais en même temps, parce que toutes les tâches prennent le pool `entrepot`, d'une seule place : elles ouvrent toutes l'entrepôt, qui n'accepte qu'un écrivain. *Théorie : `Theorie_12_DE.md`, section 4.*

7. Le fichier attaché deux fois. Le script commence par `ATTACH … AS entrepot`, parce qu'on le lance par `duckdb -f` depuis une base vide. Le pipeline tient déjà l'entrepôt ouvert comme base principale, et DuckDB refuse d'attacher une seconde fois un fichier déjà attaché, même sous un autre nom. Une seconde copie du script, sans `ATTACH`, divergerait de la première à la première retouche ; l'étape adapte donc l'unique script, et vérifie qu'elle trouve exactement ce qu'elle retire. *Théorie : `Theorie_12_DE.md`, section 4.*

8. Le pilote dans l'image. Ce qui s'installe au démarrage se réinstalle à chaque démarrage, et finit par ne plus s'installer, le jour où le dépôt de paquets ne répond pas ; posé à la construction, il est dans l'image, figé et vérifié une fois. En `root`, parce que c'est un paquet du système, que seul l'administrateur installe ; l'image rend ensuite la main à l'utilisateur `airflow`. *Théorie : `Theorie_12_DE.md`, section 4.*

© 2026 iAlexMG. Tous droits réservés.

9. Deux adresses pour une base. Sous Windows, le conteneur de la base publie son port 5432 sur le port 5433 de la machine, joignable par `localhost`. Dans le réseau des conteneurs, la base s'appelle par son nom de service, `erp`, sur son port interne, 5432. Le code ne change pas : `config.toml` donne `localhost` et 5433, et le compose surcharge les deux pour les conteneurs d'Airflow par `PIPELINE_ERP_SERVEUR` et `PIPELINE_ERP_PORT`, la surcharge par l'environnement du Cours 05. *Théorie : `Theorie_12_DE.md`, section 4.*

10. Le fuseau de la session. La fenêtre portait sur `heure`, un instant absolu, comparé à des dates. DuckDB lit alors l'instant dans le fuseau de la session : UTC dans le conteneur. Les quatre dernières heures du 31 mars, de 20 h à minuit à l'heure de l'Est, sont le 1er avril en UTC : elles tombent dans la fenêtre d'avril, qui compte un jour présent, donc 29 manquants. *Théorie : `Theorie_12_DE.md`, section 4.*

11. Un chiffre du milieu. Ni la règle neuve ni la fenêtre corrigée ne changent ce qu'un consommateur lit : les tables, leurs colonnes et leur sens restent les mêmes. La règle ajoute une promesse, la fenêtre corrige la façon de juger. Le premier chiffre monte quand il faut toucher au code qui lit (Cours 11) ; ici, le deuxième suffit. *Théorie : `Theorie_12_DE.md`, section 4.*

12. Trois preuves. Les comptes : chaque table contre `LIVRAISON.md`, par le kit ; ils voient une table absente ou un fichier chargé deux fois, pas deux lignes échangées au même total. Les lignes : chaque table refaite contre celle d'avant, par `EXCEPT ALL` dans les deux sens ; elles voient la moindre valeur qui diffère, pas ce qui était déjà faux avant. Les contrats : les 23 règles sur tout l'historique ; ils voient ce que la donnée promet, pas une promesse que personne n'a écrite. *Théorie : `Theorie_12_DE.md`, section 5.*

13. Mettre de côté. La deuxième preuve compare le refait à l'entrepôt d'avant, ligne par ligne : il faut qu'il existe encore. Déplacé, il sert de témoin ; supprimé, il ne reste que les comptes du kit et les contrats, qui ne voient pas deux lignes échangées contre deux autres. *Théorie : `Theorie_12_DE.md`, section 5.*

14. La même minute. L'alerte juge le temps de l'événement, l'instant que porte chaque point, et non l'heure à laquelle il arrive (Cours 09). Rejouer plus tôt, plus tard, plus vite ou ailleurs ne change pas les instants des points, donc pas la fenêtre où la moyenne franchit 4,5 mm/s. Seule `recu_le`, l'heure de réception, change : elle dit quand le rejeu est passé. *Théorie : `Theorie_12_DE.md`, section 6.*

15. Qui fait quoi dans Grafana. Data Engineering livre la plomberie et la donnée : la source déclarée par fichier, le tableau de bord écrit dans le dépôt, et des tables du temps réel justes. Business Intelligence conçoit la lecture : ce qu'un lecteur voit d'abord, pour quel public, avec quels seuils visibles. `allowUiUpdates: false` interdit d'enregistrer depuis l'interface une retouche d'un tableau de bord venu d'un fichier : Grafana répond `Cannot save provisioned dashboard`, et la retouche se fait dans le fichier. *Théorie : `Theorie_12_DE.md`, section 7.*

Questions pratiques

1. [labo] Le coût de la traversée.

```sql
SELECT 'avant' AS entrepot, etat, lignes, ROUND(duree_s, 1) AS duree_s
FROM avant.pipeline.executions WHERE etape = 'argent'
UNION ALL
SELECT 'refait', etat, lignes, ROUND(duree_s, 1)
FROM refait.pipeline.executions WHERE etape = 'argent' ORDER BY 1;
```


© 2026 iAlexMG. Tous droits réservés.

entrepot varchar	etat varchar	lignes int64	duree_s double
avant	réussi	290567	12.0
refait	réussi	290567	105.1

Le même script, les mêmes 290 567 lignes, près de neuf fois plus lent dans le conteneur. La première ligne est le passage de l'étape 1, sous Windows, qui a écrit dans l'entrepôt d'avant ; la seconde, celui de la reconstruction, dans un conteneur qui lit le bronze à travers le montage de WSL 2. C'est la lecture des fichiers qui coûte, pas le calcul. *Théorie : `Theorie_12_DE.md`, section 4.*

2. [labo] L'histoire que le journal garde.

```
SELECT 'avant' AS entrepot, table_raw, fichier, COUNT(*) AS versions,
       STRING_AGG(lignes::VARCHAR, ' puis ' ORDER BY charge_le) AS lignes
FROM avant.pipeline.chargements GROUP BY ALL HAVING COUNT(*) > 1
UNION ALL
SELECT 'refait', table_raw, fichier, COUNT(*),
       STRING_AGG(lignes::VARCHAR, ' puis ' ORDER BY charge_le)
FROM refait.pipeline.chargements GROUP BY ALL HAVING COUNT(*) > 1 ORDER BY 1, 2;
```

entrepot varchar	table_raw varchar	fichier varchar	versions int64	lignes varchar
avant	corpus_index	sources/corpus/index.csv	2	33 puis 34
avant	usine_bons_travail	sources/usine/gmao/bons_travail.csv	2	779 puis 825

Deux fichiers chargés deux fois dans l'entrepôt d'avant, au Cours 03 puis au Cours 06 ; aucun dans le refait, qui n'a connu que la génération jusqu'à mars 2026. La `RAW`, elle, ne garde que la dernière version dans les deux entrepôts : un fichier livré de nouveau sous le même nom remplace ses lignes (Cours 03). *Théorie : `Theorie_12_DE.md`, section 3.*

3. [labo] La fraîcheur de l'argent, mois par mois. Depuis `01_de` :

```
python rejouer_contrats.py argent --depuis 2025-10 --jusqua 2026-04 --par-mois
```

```
7 tranches jugées, en 3 s.
  moyenne_dans_la_plage_du_capteur         avertissement    4 hors cible : 2025-10-01 2025-11-01 2026-01-01 2026-03-01
  heures_du_jour_livrees                   bloquant         1 hors cible : 2026-04-01
  couverture_bornee                        bloquant         0 hors cible
  production_egale_au_cumul                bloquant         0 hors cible
  une_ligne_par_etiquette_et_par_heure     bloquant         0 hors cible
```


© 2026 iAlexMG. Tous droits réservés.

```
  valeur_des_qu_une_seconde_est_couverte   bloquant         0 hors cible
Alertes envoyées : 5, dont 1 bloquantes.
```

Seul avril rompt la fraîcheur : aucun mois de la génération ne manque un jour. Les dix heures hors de la plage de leur capteur se répartissent sur quatre mois, octobre, novembre, janvier et mars. *Théorie : `Theorie_12_DE.md`, section 4.*

4. [labo] Les jours qui n'ont pas 24 heures.

```sql
SELECT CAST(heure AT TIME ZONE 'America/Toronto' AS DATE) AS jour_boreal,
       COUNT(DISTINCT heure) AS heures
FROM refait.argent.usine_horaire
WHERE heure AT TIME ZONE 'America/Toronto' >= TIMESTAMP '2026-03-01'
GROUP BY 1 HAVING COUNT(DISTINCT heure) <> 24 ORDER BY 1;

SELECT CAST(heure AT TIME ZONE 'UTC' AS DATE) AS jour_utc, COUNT(DISTINCT heure) AS heures
FROM refait.argent.usine_horaire
WHERE heure AT TIME ZONE 'UTC' >= TIMESTAMP '2026-03-01'
GROUP BY 1 HAVING COUNT(DISTINCT heure) <> 24 ORDER BY 1;
```

jour_boreal date	heures int64
2026-03-08	23

jour_utc date	heures int64
2026-04-01	4

En jours de Boréal, un seul écart : le 8 mars, passage à l'heure avancée, compte 23 heures (Cours 08). En jours UTC, le changement d'heure disparaît, UTC n'en a pas, mais un 1er avril de quatre heures apparaît : les quatre dernières heures du 31 mars à l'heure de l'Est. C'est exactement ce qui faussait la fenêtre du contrat dans le conteneur. *Théorie : `Theorie_12_DE.md`, section 4.*

5. [labo] Le journal sans son état.

```sql
SELECT fichier, table_raw, lot, lignes FROM (
  FROM (SELECT * EXCLUDE (charge_le) FROM avant.pipeline.chargements)
  EXCEPT ALL
  FROM (SELECT * EXCLUDE (charge_le) FROM refait.pipeline.chargements)) ORDER BY 1;
```


© 2026 iAlexMG. Tous droits réservés.

```
SELECT COUNT(*) AS surplus FROM (
  FROM (SELECT * EXCLUDE (charge_le) FROM refait.pipeline.chargements)
  EXCEPT ALL
  FROM (SELECT * EXCLUDE (charge_le) FROM avant.pipeline.chargements));
```

fichier varchar	table_raw varchar	lot varchar	lignes int64
sources/corpus/index.csv	corpus_index	complet	33
sources/usine/gmao/bons_travail.csv	usine_bons_travail	complet	779

surplus int64
0

Deux lignes dans le sens « avant moins refait », les premières versions de la GMAO et du catalogue ; aucune dans l'autre sens. Ce ne sont pas des défauts de la reconstruction : ce sont des faits d'histoire que les sources ne portent plus. D'où `ETAT` : on compare l'état du journal, et l'on compte l'histoire à part. *Théorie : `Theorie_12_DE.md`, sections 3 et 5.*

6. [labo] Le volume de points, vu du chemin chaud.

```
SELECT instant::date AS jour, count(*) AS points
FROM temps_reel.points
WHERE etiquette = 'UTL.AIR.PRESSION' AND instant < '2025-10-25'
GROUP BY 1 ORDER BY 1;
```

```
    jour    | points
------------+--------
 2025-10-13 |     24
 2025-10-14 |     92
 2025-10-15 |     93
 2025-10-16 |     92
 2025-10-17 |    105
 2025-10-18 |     24
 2025-10-19 |     24
 2025-10-20 |    198
 2025-10-21 |    239
 2025-10-22 |    232
 2025-10-23 |    259
```


© 2026 iAlexMG. Tous droits réservés.

```
 2025-10-24 |    265
(12 rows)
```

Environ 92 points les jours de production avant la dérive, 24 les jours sans production, un point par heure à l'archivage forcé. Le 17 monte déjà à 105, puis le volume double le 20. La bande morte de l'historien écrit un point chaque fois que la valeur bouge assez (Cours 08) : une pression qui oscille écrit plus. C'est la règle de volume que le Cours 11 suggérait, lue cette fois dans le chemin chaud. *Théorie : `Theorie_12_DE.md`, section 6.*

7. [labo] Où passe le temps d'Airflow.

```sql
WITH r AS (SELECT * FROM refait.pipeline.executions ORDER BY debut LIMIT 7)
SELECT COUNT(*) AS etapes, ROUND(SUM(duree_s), 1) AS somme_s,
       ROUND(EPOCH(MAX(fin) - MIN(debut)), 1) AS etendue_s,
       ROUND(100 * SUM(duree_s) FILTER (WHERE etape = 'argent') / SUM(duree_s)) AS part_argent
FROM r;
```

etapes int64	somme_s double	etendue_s double	part_argent double
7	183.1	190.4	57.0

Le travail du pipeline tient en 183 secondes ; entre le début du bronze et la fin de la météo, il s'en écoule 190 ; Airflow donne à l'exécution 191,8 secondes, du 17:27:13 au 17:30:24 dans le labo. Les sept secondes d'écart entre les deux premiers chiffres sont le coût d'Airflow entre deux tâches : l'ordonnanceur voit la fin de l'une, lance un processus pour la suivante, qui démarre Python et importe le pipeline, un peu plus d'une seconde chaque fois. L'argent, lui, pèse 57 % du travail. *Théorie : `Theorie_12_DE.md`, section 4.*

8. [labo] Le bulletin a-t-il été suivi ?

```sql
SELECT type_visite, date_visite >= '2026-02-16' AS apres_bulletin,
       COUNT(DISTINCT no_visite) AS visites, COUNT(DISTINCT sernr) AS motoneiges
FROM refait.raw.garanties_visites
WHERE sernr BETWEEN 'MN26001238' AND 'MN26001466'
GROUP BY ALL ORDER BY ALL;
```

type_visite varchar	apres_bulletin boolean	visites int64	motoneiges int64
BULLETIN_SERVICE	true	210	210
PREMIER_ENTRETIEN	false	69	69
PREMIER_ENTRETIEN	true	62	62

© 2026 iAlexMG. Tous droits réservés.

Deux cent dix des 229 motoneiges de la plage ont eu leur visite d'inspection du bulletin, toutes après le 16 février. La réponse dit aussi ce qu'elle ne peut pas dire : si les réclamations retombent en mars à cause de ces visites, c'est une question d'analyse, qui demande de joindre les visites aux réclamations, et elle appartient à Analytics Engineering et à Data Science. *Théorie : `Theorie_12_DE.md`, section 8.*

9. [labo] La fenêtre de la météo. Les trois nombres viennent de la section `[meteo]` de `01_de/config.toml` : `debut = "2025-10-01"`, `fin = "2025-12-31"`, soit 92 jours, et `page = 50`, soit deux pages, 50 jours puis 42. La tâche a écrit `plateforme/lac/bronze/meteo_api/quotidien_7060400_2025-10-01_2025-12-31.jsonl`, la station 7060400 et la fenêtre dans le nom. C'est une valeur qui change d'un environnement à l'autre, rangée hors du code par le Cours 05 ; la reconstruction la relit telle quelle. *Labo : `Labo_12_DE.md`, étape 5.*

10. [labo] Un panneau de plus. La requête du panneau, avec la macro de période de Grafana :

```sql
SELECT time_bucket('1 day', instant, 'America/Toronto') AS time, count(*) AS "points de pression"
FROM temps_reel.points
WHERE etiquette = 'UTL.AIR.PRESSION' AND $__timeFilter(instant)
GROUP BY 1 ORDER BY 1
```

Vérifiée dans `psql`, la macro remplacée par `instant >= '2025-10-18' AND instant < '2025-10-23'`, elle rend 24, 24, 198, 239 et 232 points, datés de minuit à l'heure de l'Est, du 18 au 22 : les comptes de la question 6. Sans son troisième argument, `time_bucket('1 day', instant)` coupe les jours à minuit UTC, 20 h à l'heure de l'Est : six tranches au lieu de cinq, de 20, 24, 198, 239, 232 et 4 points, et les 198 points du 20 octobre s'affichent au 19 octobre, 20 h. Le piège du fuseau, une fois de plus, mais dans un graphique. *Théorie : `Theorie_12_DE.md`, section 7.*

Diagnostics

1. Il peut encore prouver les comptes, par le kit et `--verifier`, et les contrats, qui se rejugent sur le refait seul. Il ne peut plus prouver les lignes : il n'y a plus rien à comparer. Une reconstruction qui rendrait les bons totaux avec deux lignes échangées passerait. Un entrepôt supprimé ne se retrouve pas ; il faudrait refaire l'épreuve, en déplaçant cette fois. *Théorie : `Theorie_12_DE.md`, section 5.*

2. Le kit rejoue `python -m pipeline tout`, puis refait les séries lui-même par `argent_usine.sql` : avec `argent` dans « tout », les séries se font deux fois, une douzaine de secondes perdues à chaque passage, sans faute de résultat. Et « tout » change de sens : depuis le Cours 05, c'est le chemin qui charge ce qui arrive, bronze, `RAW`, corpus, celui que le kit rejoue avant de refaire les séries. La docstring de `__main__.py` le dit en toutes lettres, et c'est elle qu'il faudrait réécrire d'abord. *Théorie : `Theorie_12_DE.md`, section 4.*

3. Sans pool, les deux tâches sans flèche partent dès le début de l'exécution, en même temps que `bronze`, et les trois ouvrent l'entrepôt, puisque chaque étape y inscrit son passage. Celle qui arrive la deuxième trouve le fichier verrouillé, `Could not set lock on file`, et le pipeline s'arrête sur « L'entrepôt est occupé » : le piège du Cours 06, un seul écrivain à la fois sur le fichier DuckDB. Le pool ne ralentissait rien d'utile : le goulot est l'entrepôt, pas le nombre de tâches. *Théorie : `Theorie_12_DE.md`, section 4.*

4. La correction de la collègue règle le fuseau d'une machine, pas celui du contrat. Le jour où le contrat tourne ailleurs, un autre ordonnanceur, un serveur en UTC, une plateforme infonuagique, un collègue à Vancouver, il rend de nouveau d'autres verdicts. Le contrat dit « le jour de Boréal » : c'est dans la règle qu'il faut l'écrire, `heure AT TIME ZONE 'America/Toronto'`, pour qu'elle juge pareil partout. *Théorie : `Theorie_12_DE.md`, section 4.*

© 2026 iAlexMG. Tous droits réservés.

5. Tout ce que le refait porte en trop : un fichier chargé deux fois par la reconstruction, des lignes en double, une table qui aurait gagné des lignes d'un autre lot. `avant EXCEPT ALL refait` ne voit que ce qui manque. Une reconstruction qui double une livraison passerait, et c'est précisément le défaut que l'idempotence du Cours 03 empêche, et qu'une preuve doit voir. *Théorie : `Theorie_12_DE.md`, section 5.*

6. La copie vit dans le volume de Grafana, pas dans le dépôt : elle n'a ni historique ni relecture, et elle disparaît avec le volume, au prochain changement de serveur. Deux tableaux de bord portent maintenant presque le même nom, et personne ne sait lequel fait foi. Et la retouche n'arrive jamais dans `usine.json`, si bien que le prochain Grafana neuf revient à l'ancien. La bonne voie : proposer la retouche dans le fichier, où elle se relit et se versionne. *Théorie : `Theorie_12_DE.md`, section 7.*

7. Les points sont déjà dans `temps_reel.points`, à la même étiquette et au même instant : la clé primaire les écarte, et le consommateur les compte tous en « déjà présents », aucun inséré. L'alerte, elle, se lève quand même le 22 octobre à 8 h 15 : le programme calcule sur le flux de Kafka, pas sur la base, et ses quatre changements d'état reviennent en « déjà présents » à leur tour. Rien n'est doublé, grâce à la base ; mais le rejeu ne prouve plus rien, puisque rien n'y est arrivé. *Théorie : `Theorie_12_DE.md`, section 6.*

8. Deux choses. La moyenne de `time_bucket` est une moyenne simple des points de la fenêtre : 4,570 à 7 h 45, sur deux points. L'alerte du Cours 09 pondère chaque valeur par le temps qu'elle a tenu, et seulement pendant la marche du compresseur ; sur la même fenêtre, elle reste sous 4,5, et ne passe au-dessus qu'à la fenêtre de 8 h, 4,545. Une moyenne simple de points écrits par bande morte mesure la cadence d'écriture autant que la grandeur (Cours 08). Et la fenêtre de 7 h 45 se ferme à 8 h : ce n'est pas un quart d'heure d'avance, c'est un autre calcul. *Théorie : `Theorie_12_DE.md`, section 6.*

Questions à développer

A. La sauvegarde et la reconstruction ne répondent pas à la même question. La sauvegarde garde un état et son histoire : les versions remplacées du journal, les exécutions passées, les verdicts d'hier, tout ce que la reconstruction du capstone a perdu. Mais elle ne dit rien de la plateforme qui l'a produit : restaurée, elle rend l'entrepôt, pas la capacité de le refaire. La reconstruction, elle, prouve que le dépôt, les livraisons et la plateforme suffisent. Et elle trouve des défauts qu'aucune sauvegarde ne montre : au capstone, un script qui attachait deux fois l'entrepôt, un pilote absent de l'image, un contrat qui jugeait autrement en UTC, un service qui retenait le lac effacé. Tous dormaient tant que la plateforme restait sur sa machine. Il faut donc les deux : la sauvegarde pour l'histoire, la reconstruction pour la preuve, faite avant d'en avoir besoin, pas le jour de la panne. *Théorie : `Theorie_12_DE.md`, sections 1 et 3.*

B. Le chemin froid part de fichiers livrés et gardés : relancer les étapes relit les mêmes fichiers, et rend le même résultat. Le chemin chaud n'a pas de fichier : il a ce qui passe, et ce qui est passé ne repasse pas. Le capstone le rejoue parce qu'un rejoueur tient lieu d'usine, depuis ce que le générateur a produit. Kafka garde ses messages pendant sa rétention, sept jours par défaut : pendant sept jours, un groupe neuf peut relire tout le flux, et reconstruire la base du temps réel sans rejoueur ; au-delà, les messages sont effacés. Pour rejouer l'usine d'il y a deux ans, il faut l'avoir gardée ailleurs : c'est le rôle de l'archive que l'abonné écrit (Cours 09), ou du bronze de l'historien, livré en fichiers. Le chemin chaud se reconstruit donc à partir du chemin froid, et c'est une raison de plus de garder les livraisons. *Théorie : `Theorie_12_DE.md`, section 6.*

C. Data Engineering doit à l'analyse trois choses : que chaque fait soit arrivé, complet et juste, ce que prouvent les contrats ; que chacun soit rangé là où une autre formation sait le lire, la `RAW`, l'argent, le temps réel ; et que chacun garde son lignage, le fichier et le lot d'où il vient. Il s'arrête avant de joindre et d'interpréter : relier un numéro de série à ses réclamations, décider qu'un compresseur a causé une surchauffe, c'est construire un modèle du métier, et ce

© 2026 iAlexMG. Tous droits réservés.

modèle appartient à Analytics Engineering, puis à Data Science. S'il le faisait, la `RAW` cesserait d'être le brut, et l'analyse perdrait la possibilité de refaire le raisonnement autrement. Les requêtes du labo filtrent chaque table par la plage, et c'est tout : elles prouvent que la matière est là, pas ce qu'elle veut dire. *Théorie : `Theorie_12_DE.md`, section 8.*

Mises en situation

1. Laiterie de Saint-Hyacinthe. D'abord écrire, tant que l'ingénieur est là : un `README.md` qui dit les prérequis, les commandes dans l'ordre et ce qu'elles doivent afficher ; la liste de ce qui n'est pas dans le dépôt, données, secrets, réglages du serveur ; et un document de livraison, table par table, avec les comptes attendus. Ensuite versionner ce qui ne l'est pas : tout ce qui a été réglé à la main sur le serveur, les tâches planifiées, les paquets installés, les tableaux de bord montés dans une interface. Puis faire l'épreuve, sur une autre machine si possible, et par une autre personne : mettre l'entrepôt de côté, effacer ce qui se dérive, tout refaire depuis le dépôt et les livraisons gardées, en suivant le `README.md` seulement. Chaque question posée à l'ingénieur pendant l'épreuve est une ligne qui manque au fichier. La preuve que c'est fait : les comptes attendus, les lignes comparées dans les deux sens à 0 écart, et les contrats au vert, obtenus par quelqu'un d'autre que lui. Enfin étiqueter l'état livré, pour qu'on sache à quoi revenir. *Théorie : `Theorie_12_DE.md`, sections 1 et 9.*

2. Minière de Val-d'Or. Le symptôme, une rupture une nuit sur trois sur le serveur et jamais sur les portables, désigne ce qui diffère entre les deux machines, et le fuseau en est le premier suspect. Diagnostic : rejuger le contrat sur une nuit rompue, sur le serveur et sur un portable, puis lire l'instant d'une ligne des deux côtés, avec `current_setting('TimeZone')` ; compter les heures d'un jour en UTC et à l'heure de l'Est, comme au labo. Si la fenêtre compare un instant à des dates, elle coupe les jours à minuit UTC sur le serveur, 19 h ou 20 h à Val-d'Or, et les chargements du soir changent de jour. Correction : écrire le jour du site dans la règle, `AT TIME ZONE 'America/Toronto'`, rejouer le contrat sur tout l'historique des deux côtés, et passer au deuxième chiffre de sa version. Ne pas régler le fuseau du serveur : la règle doit juger pareil partout. *Théorie : `Theorie_12_DE.md`, section 4.*

3. Coopérative de Gaspé. L'export la veille marche une fois, et encore : il dépend d'une personne qui y pense, et la version exportée n'a ni historique ni relecture. Proposition : exporter une dernière fois, puis faire de cet export un fichier du dépôt, relu par une deuxième personne ; déclarer par fichier la source de données et le dossier des tableaux de bord, dans la configuration du nouveau serveur ; et interdire l'enregistrement depuis l'interface. Le changement de serveur devient alors une reconstruction ordinaire : le nouveau Grafana relit les fichiers, et le tableau de bord revient à l'identique. L'analyste garde la main sur la lecture, les panneaux et les seuils, mais ses retouches passent par le fichier. *Théorie : `Theorie_12_DE.md`, section 7.*

4. (avancé) Distributeur de Joliette. Les deux écarts du journal, dans le sens « avant moins refait », ressemblent à de l'histoire : des versions remplacées d'un fichier livré de nouveau sous le même nom. Vérification : sont-ils les plus anciennes versions de fichiers dont le refait porte la dernière, à l'empreinte près ? Si oui, ils se comptent à part et ne sont pas des écarts. L'écart de la table des factures est d'une autre nature : une ligne que le refait porte en trop. C'est un vrai défaut jusqu'à preuve du contraire : un fichier chargé deux fois, un doublon que l'ancien chargement écartait, une règle de chargement qui a changé depuis. Vérification : la ligne, son lignage, son fichier et son lot, et la même clé dans l'entrepôt d'avant. Réponse à qui demande : la reconstruction n'est pas prouvée tant que la ligne en trop n'est pas expliquée ; les deux lignes du journal, elles, le sont, et le rapport doit les montrer à part plutôt que de les mêler aux écarts. *Théorie : `Theorie_12_DE.md`, section 5.*

© 2026 iAlexMG. Tous droits réservés.